

Мега исследование по построению моделей

Изучение существующих методов аппроксимации

Для выбора оптимального подхода, проведём обзор основных классов методов, которые потенциально применимы к нашему кейсу:

1. Линейная и полиномиальная регрессия

- *Суть:* Модель пытается представить целевую метрику как сумму входных признаков с определёнными весами
- *Плюсы и минусы:* Быстро обучается, легко интерпретируется, но чистая линейная модель не может улавливать сложные нелинейные связи без ручного преобразования

2. Древоидные ансамбли (Random Forest, Gradient Boosting)

- *Суть:* Алгоритмы строят ансамбль решающих деревьев, где каждое дерево принимает решение на основе последовательных условий
- *Плюсы и минусы:* Отлично работают с нелинейными данными и пороговыми значениями, устойчивы к выбросам. Плохо предсказывать значения за пределами границ обучающей выборки

3. Методы опорных векторов (SVR)

- *Суть:* Переносит данные в более высокое пространство размерностей, где пытается построить разделяющую гиперплоскость
- *Плюсы и минусы:* Хорошо работает на небольших выборках, но долго обучается при росте количества данных и требует сложной настройки параметров

4. Нейронные сети

- *Суть:* Набор связанных слоёв нейронов с нелинейными функциями активации
- *Плюсы и минусы:* Могут выучить любую зависимость, но требуют очень много данных для обучения

Выбор методов для экспериментов и их обоснование

На основе анализа специфики задачи для проведения экспериментов были выбраны два метода: линейная регрессия с регуляризацией L2 (Ridge) в качестве базового уровня (baseline) и ансамбль решающих деревьев (Random Forest) в качестве целевой нелинейной модели. Переход к анализу сложной модели обосновывается системой критериев достаточности линейного приближения.

Линейная (Ridge) модель считается математически недостаточной для аппроксимации характеристик файловой системы, если выполняется хотя бы одно из следующих условий:

1. **Критерий детерминации:** R^2 на тестовой выборке < 0.7 хотя бы для одной из четырёх целевых системных метрик
2. **Критерий структуры остатков:** Наличие выраженной систематической структуры или гетероскедастичности в остатках (вместо распределения в виде случайного гауссова шума вокруг нулевой линии на графике «остатки vs предсказание»)

Обоснование выбора:

• Ожидаемая форма зависимостей

Из теории алгоритмов и структуры исследуемых подходов (A, B, C) можно выделить следующие особенности:

- **Нелинейность:** В подходе A время поиска зависит от глубины D и ширины W как $O(D \cdot \log W)$ или $O(D \cdot W)$. Это явно нелинейная зависимость. Чтобы линейная регрессия могла её уловить, мы применим логарифмирование признаков

- **Экспоненциальный рост:** В подходах В и С групповые операции вроде `ls` или `find` зависят от размера поддерева, который при увеличении коэффициента заполнения F может расти экспоненциально
- **Количество доступных данных**
Согласно плану формирования сетки, мы имеем:

$$N \approx 1000 \text{ точек измерений на один подход}$$

- **Требования к интерпретируемости**
 - **Линейная регрессия** даёт максимальную интерпретируемость: знак и величина коэффициента перед признаком показывают, насколько сильно и в какую сторону меняется метрика при изменении признака
 - **Random Forest** менее прозрачен, но он предоставляет оценку важности признаков. С помощью неё мы сможем построить график и наглядно показать, какой из параметров вносит наибольший вклад

- **Вычислительные ресурсы**

Поскольку обученные модели будут использоваться внутри рекомендателя, хотелось бы иметь хорошую скорость на этапе предсказания

- Линейная регрессия выполняет только операцию скалярного произведения, что работает мгновенно
- Обученный Random Forest тратит время только на проход по веткам деревьев фиксированной глубины, что также имеет константную сложность $O(1)$ относительно объема данных

Оба метода легковесны и не потребуют мощных вычислительных ресурсов ни при обучении, ни при работе рекомендателя.

- **Наличие корреляций между целевыми метриками**

Исходя из физики работы вычислительных систем, между нашими целевыми метриками ожидается сильная взаимосвязь:

- Пропускная способность (Throughput) и средняя латентность (Latency) связаны обратно пропорционально, чем выше задержки, тем меньше операций в секунду обрабатывает система
- 99-й перцентиль латентности очевидно связан со средней латентностью
- Потребление памяти (Memory) теоретически не должно напрямую зависеть от временных характеристик. В основном будет зависеть от структуры данных (D, W, F)

Проектирование признаков

Чтобы повысить точность аппроксимации и помочь линейной модели лучше улавливать особенности файловых систем, был проведён синтез новых нелинейных признаков. Базовый набор параметров включает в себя: глубину дерева (D), ширину папки (W), общее количество файлов (F) и относительную частоту вызова конкретных операций op_i

Математическое обоснование новых признаков

1. Логарифмирование структурных параметров ($\log D, \log W$)

Входные параметры структуры данных могут различаться на несколько порядков. Из-за этого линейная регрессия начинает сильно ошибаться на краях выборки. Так же известно, что время поиска элемента в структуре часто зависит от параметров нелинейно (например, $O(D \cdot \log W)$). Это позволяет свести мультипликативные эффекты к аддитивным

2. Конструирование признаков (Feature Engineering)

Для компенсации нелинейности базовый вектор параметров расширен синтетическими признаками:

- **Геометрия структуры:**

- `estimated_nodes` - оценка числа узлов дерева через сумму геометрической прогрессии:

$$\text{estimated_nodes} = F \cdot \sum_{i=0}^D W^i = F \cdot \frac{W^{D+1} - 1}{W - 1}$$

- `log_estimated_nodes = ln(1 + estimated_nodes)` - стабилизирует дисперсию

- **Эффекты локальности и перекрестные связи:**

- `zipf_depth_interaction (zipf_s · ln(D))` - усиление эффекта неравномерности доступа с ростом глубины. При `distribution == uniform` параметру `zipf_s` принудительно присваивается 0.0.
- `nodes_x_read (estimated_nodes ·  $p_{\text{read}}$ )` - затраты на удержание «холодных» узлов при поиске.
- `nodes_x_mutation (estimated_nodes · ( $p_{\text{write}} + p_{\text{mv}}$ ))` - нагрузка от мутации больших деревьев (каскадные аллокации).
- Кодирование `distribution` через One-Hot Encoding (`drop_first=True`) выделяет бинарный признак `distribution_zipf`.

Анализ мультиколлинеарности и применение VIF

Поскольку признаки математически связаны, возникает проблема мультиколлинеарности (сильной линейной зависимости между предикторами). Для линейной регрессии это неприятно: веса модели становятся нестабильными

Для выявления сильно зависимых признаков мы будем использовать фактор инфляции дисперсии (VIF). Для каждого признака x_j он рассчитывается по формуле:

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

где R_j^2 - коэффициент детерминации линейной модели, в которой признак x_j выступает в роли целевой переменной, а все остальные признаки как в предикторы

Построение и обучение моделей регрессии

Масштабирование данных

Входные признаки имеют совершенно разные масштабы: объём памяти измеряется в байтах, коэффициент заполнения варьируется от 0 до 1, а логарифмы в единицах. Линейная регрессия чувствительна к масштабу, переменные с большими абсолютными значениями будут оказывать более сильное влияние на результат

Для устранения этого эффекта применяется стандартизация (Z-преобразование):

$$z = \frac{x - \mu}{\sigma}$$

где μ - среднее значение признака, а σ - стандартное отклонение

Использование цепочек регрессоров

В рамках исследования необходимо предсказать вектор из четырёх взаимосвязанных целевых метрик: объём памяти, среднюю задержку, 99-й перцентиль задержки и пропускную способность

Обучение независимых моделей для каждой метрики не учитывает физические связи внутри системы, например, то, что рост латентности снижает пропускную способность. Для решения этого использован подход цепочек регрессоров

Суть заключается в последовательном обучении регрессоров, где каждый следующий шаг принимает на вход не только матрицу признаков, но и предсказания всех предыдущих моделей

В качестве базового алгоритма внутри цепочки используется регрессия с L_2 -регуляризацией

$$L_{\text{Ridge}}(w) = \frac{1}{n} \sum_{i=1}^n (y_i - w^T x_i)^2 + \alpha \|w\|^2$$

Параметр α подбирается экспериментально и отвечает за подавление весов при избыточных признаках

Кросс-валидация и оценка качества

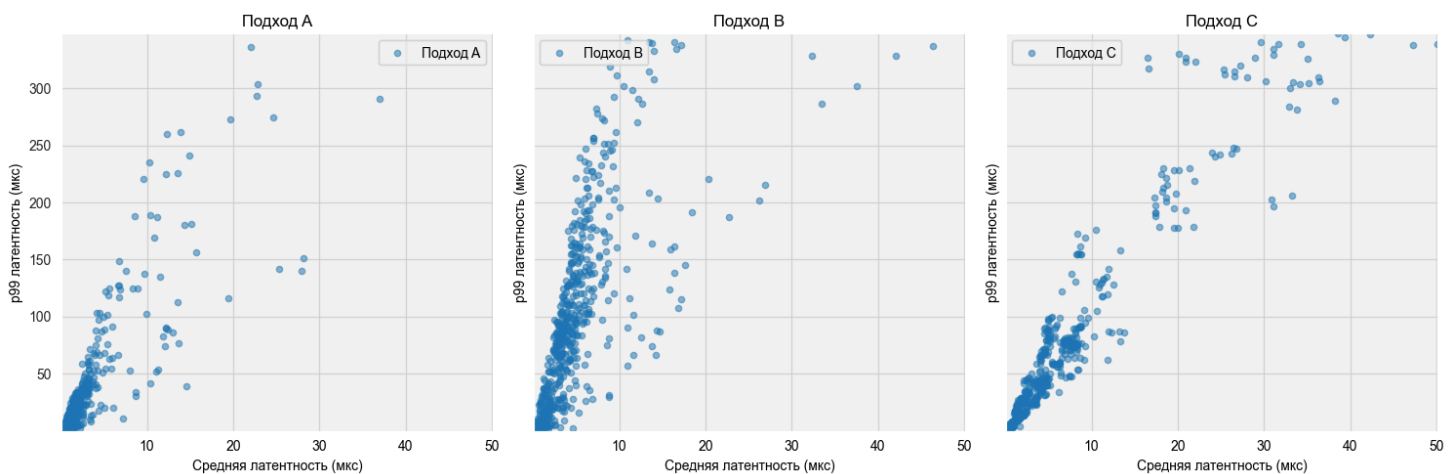
Исходный датасет разделяется в классическом соотношении: 80% данных отводится под обучение и валидацию, 20% под тестирование

Подбор гиперпараметра регуляризации α и порядок регрессоров проводятся с помощью кросс-валидации на 5 блоков. Обучающая выборка делится на 5 равных частей, модель последовательно обучается на 4 из них и проверяется на оставшейся части

Комплексный визуальный анализ экспериментальных данных

Сравнительный анализ совместного распределения задержек (p99 vs avg_latency)

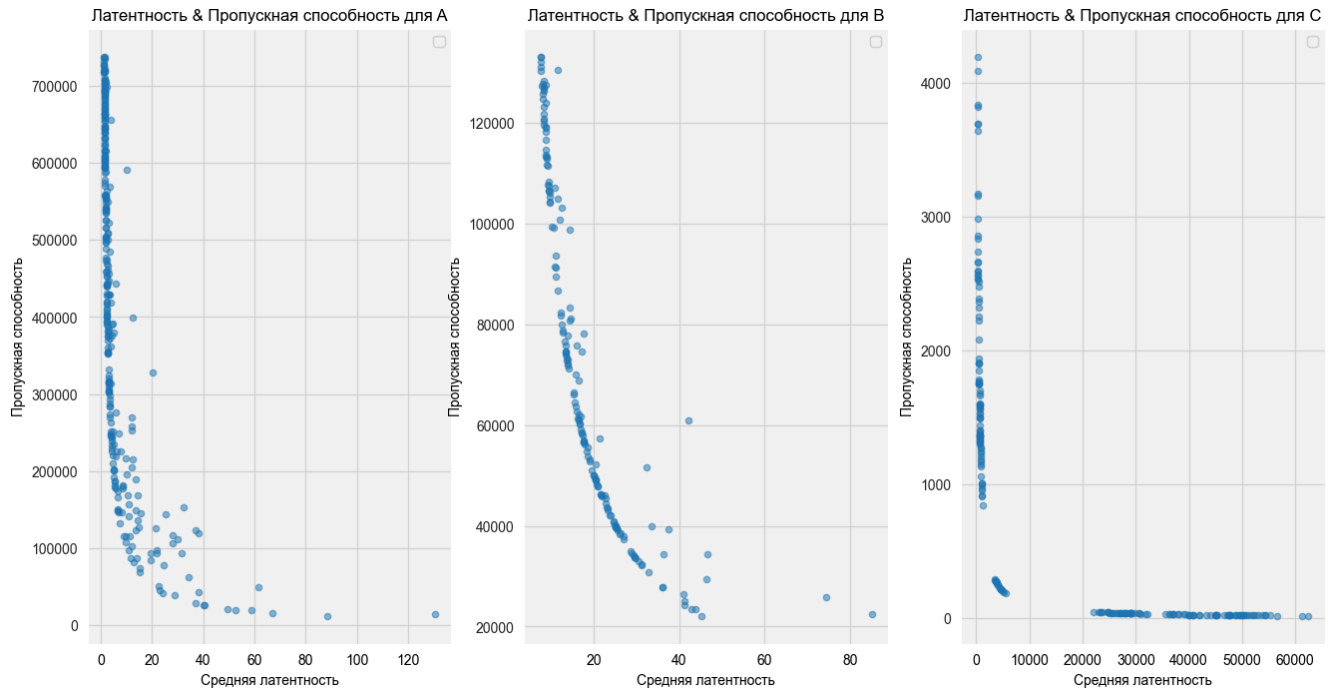
Для детального изучения структуры задержек в области стабильной работы системы (с отсечением экстремальных выбросов по порогу $\text{p99_latency_us} < 350$ мкс) были проанализированы разделенные графики рассеяния с единым масштабом осей для подходов А, В и С



- Подход А (Наивное дерево)** Большинство точек сгруппировано в безопасной зоне низких задержек (до 5 мкс по avg_latency и 50 мкс по p99). Однако при усложнении поиска линейная зависимость теряется, и график начинает сильно ветвиться. При росте средней задержки всего до 10–15 мкс происходят резкие скачки p99 вверх (до 250+ мкс). Это показывает, что наивное дерево теряет стабильность на тяжелых операциях, выдавая непредсказуемые выбросы
- Подход В (Гибрид с хэш-таблицей)** График вытянут в строгий вертикальный столб: средняя задержка намертво зажата в узком диапазоне (до 5–7 мкс), но разброс p99 огромен (до 340+ мкс). За счет поиска за $O(1)$ архитектура отлично сдерживает рост среднего времени ответа. Платой за это становятся редкие, но тяжелые операции (коллизии, рехэширование или инвалидация кэша), которые вызывают огромные всплески хвостовой латентности
- Подход С (хэш-таблица)** Здесь видна четкая ступенчатая зависимость - точки собираются в изолированные «островки» в районах 7, 18 и 33 мкс по оси X. Подход деградирует предсказу-

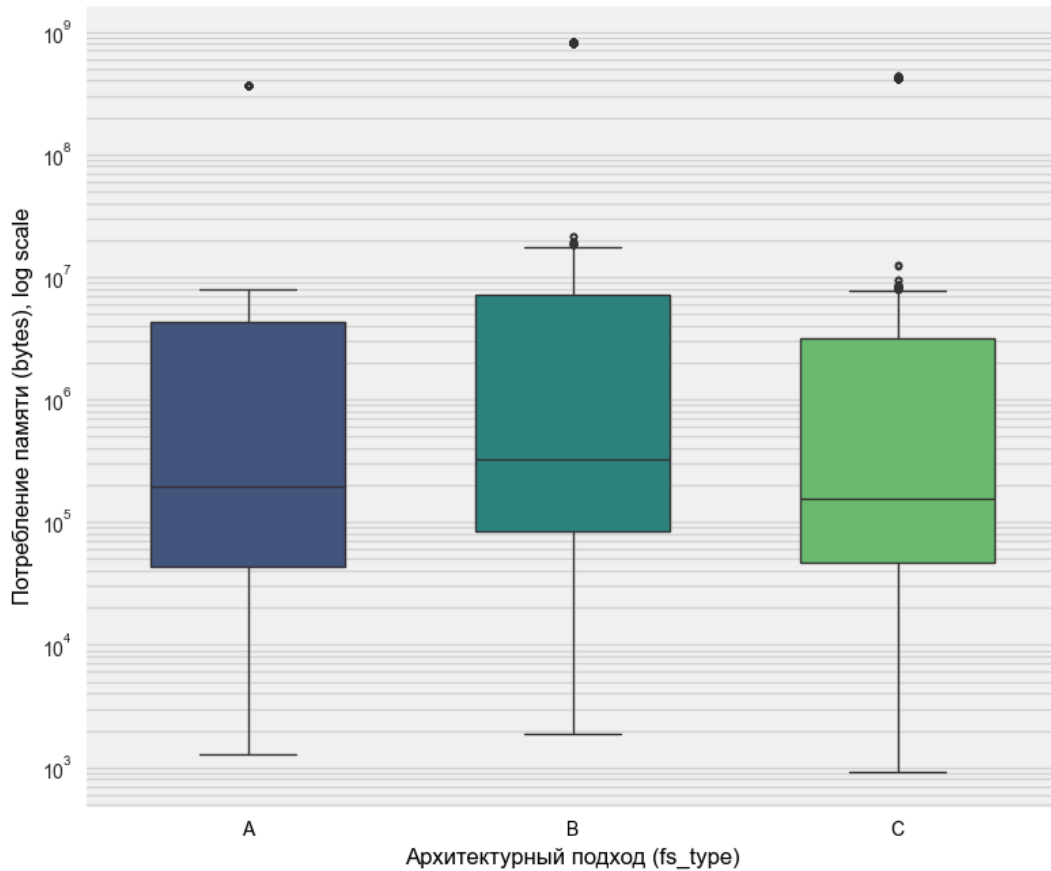
емо: r99 растет строго пропорционально средней задержке. Разделение на кластеры указывает на явные шаги в архитектуре (например, переход на новый уровень вложенности или чтение глубоких поддиректорий)

Плотности распределения пропускной способности `throughput_ops_sec`



Исследование профилей памяти

Распределение потребления памяти в зависимости от подхода и фактора заполнения (F)

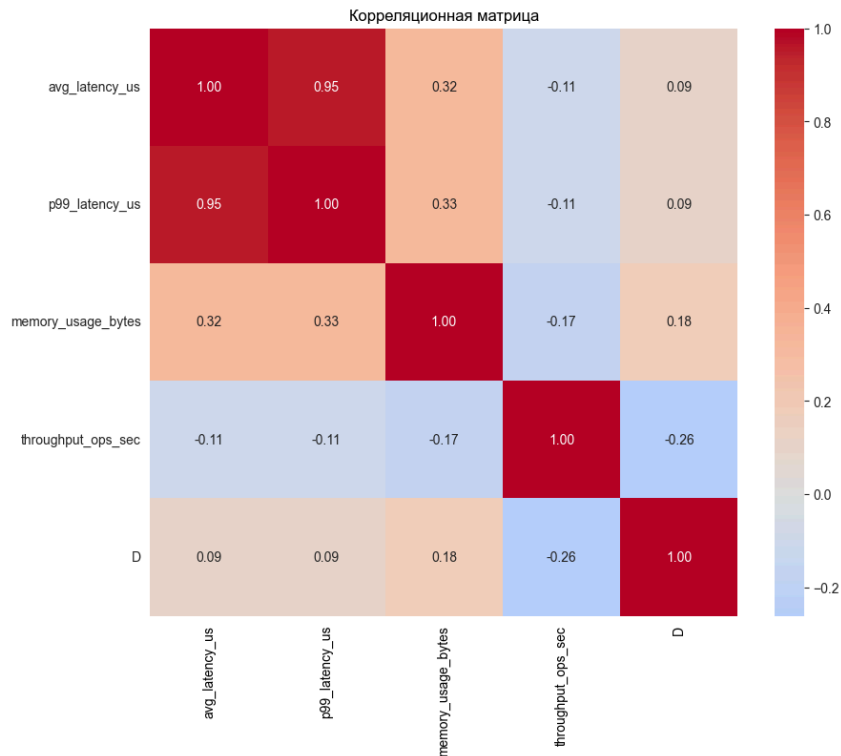


Подход В демонстрирует самые высокие показатели потребления памяти по всем ключевым метрикам: у него наибольшее медианное значение (более $3 \cdot 10^5$ байт), самый высокий верхний квартиль, а также максимальный среди всех групп единичный выброс, достигающий почти 10^9 байт

Подходы А и С имеют схожие минимальные значения (около 10^3 байт) и сопоставимые нижние границы распределения. Однако подход А имеет более высокую медиану и более широкий межквартильный размах по сравнению с подходом С, что говорит о стабильно большем потреблении памяти в типичных случаях

Подход С выглядит наиболее эффективным с точки зрения экономии ресурсов, так как его медиана находится на самом низком уровне (примерно $1.5 \cdot 10^5$ байт), а основная масса данных сдвинута ниже, чем у остальных подходов. При этом во всех трех архитектурных решениях наблюдаются единичные аномальные выбросы в районе 10^8 – 10^9 байт, что указывает на наличие специфических ресурсоемких сценариев для каждого из вариантов

Матрица корреляций



(остальные данные имеют низкую корреляцию)

Сильная линейная зависимость задержек (0.95): Между avg_latency_us и p99_latency_us наблюдается сильнейшая корреляция. Это обусловлено тем, что при общем замедлении выполнения операций предсказуемо растут и максимальные задержки

Связь потребления памяти и времени выполнения (0.32–0.33): Рост memory_usage_bytes умеренно коррелирует с увеличением латентности. Это объясняется тем, что при увеличении объема хранимых данных (например, росте количества узлов или коэффициента заполнения F) усложняется аллокация памяти и увеличивается время обхода дерева или хэш-таблиц

Обратная зависимость пропускной способности (-0.11 и -0.17): Метрика throughput_ops_sec ожидаемо демонстрирует отрицательную корреляцию с задержками и памятью. Физический смысл очевиден: чем выше вычислительная стоимость одной операции и чем больше накладные расходы на управление памятью, тем меньше операций в секунду способна обработать система

Влияние глубины дерева D: Параметр D практически не влияет на задержки напрямую (0.09). Однако он имеет слабую отрицательную корреляцию с пропускной способностью (-0.26) и положительную с памятью (0.18), что указывает на системный эффект: увеличение вложенности путей незначительно снижает темп обработки операций и увеличивает накладные расходы на хранение данных

Перейдем к рассмотрению конкретных подходов, используя выше упомянутые методы анализа и обучения

Анализ подхода А

Подход А использует наивное N -арное дерево, где каждый узел хранит массив ссылок на свои дочерние элементы. Это классическая структура, которая обеспечивает интуитивную реализацию операций, но требует последовательного обхода дерева сверху вниз для любого действия, из-за чего глубинная адресация замедляется до $O(D)$

Инженерия признаков

Для подхода А сформирован набор из 14 признаков, включающий геометрические параметры структуры и их логарифмы ($\log D$, $\log W$), полный вероятностный профиль операций (p_{ls} , p_{find} , p_{write} , p_{mv} , p_{read}), составные взаимодействия масштаба структуры с типом операции ($nodes_x_read$, $nodes_x_mutation$), синтетическую оценку масштаба ($estimated_nodes$, $\log_estimated_nodes$), параметры распределения Zipf ($zipf_s$, $zipf_depth_interaction$) и метрику локальности доступа ($locality$)

Расчёт VIF выявил коллинеарность ряда параметров, обусловленную спецификой признакового пространства:

Признак	VIF	Признак	VIF
p_{ls}	15.11	$locality$	5.89
$estimated_nodes$	13.93	$\log_estimated_nodes$	5.67
$zipf_s$	9.70	p_{read}	5.01
p_{find}	8.77	$nodes_x_read$	4.27
p_{write}	8.09	$\log W$	3.37
$\log D$	7.34	p_{mv}	3.21
$zipf_depth_interaction$	6.73	$nodes_x_mutation$	6.41

Порог $VIF > 10$ превышают два признака: p_{ls} (15.11) и $estimated_nodes$ (13.93). Причина их коллинеарности различна: p_{ls} связана с остальными долями операций через ограничение суммы к единице - строгую функциональную зависимость, характерную для любого набора, где доли операций заданы явно. $estimated_nodes$ пересекается с $\log_estimated_nodes$ и произведениями $nodes_x_read/nodes_x_mutation$ как их нелинейная основа. Признаки со значениями VIF в диапазоне 3-10 умеренно коллинеарны из-за синхронного изменения параметров в дизайне сетки экспериментов, однако ни один из них не является точной линейной комбинацией другого, поэтому набор сохранён без чистки.

Фиксация базовой модели (Ridge)

На основе 5-блочной кросс-валидации (KFold) и перебора параметров в цепочке RegressorChain была выбрана конфигурация:

$$\alpha = 0.8,$$

порядок звеньев $avg_latency_us \rightarrow throughput_ops_sec \rightarrow memory_usage_bytes \rightarrow p99_latency_us$. Выбор максимального $\alpha = 0.8$ обусловлен необходимостью стабилизации весов в условиях зафиксированной мультиколлинеарности. Выбранный порядок цепочки отражает физику процессов файловой системы: средняя задержка напрямую определяет пропускную способность, память зависит от геометрии, а наиболее нелинейная метрика хвостовой латентности (p99) замыкает цепочку, агрегируя предсказания предыдущих звеньев

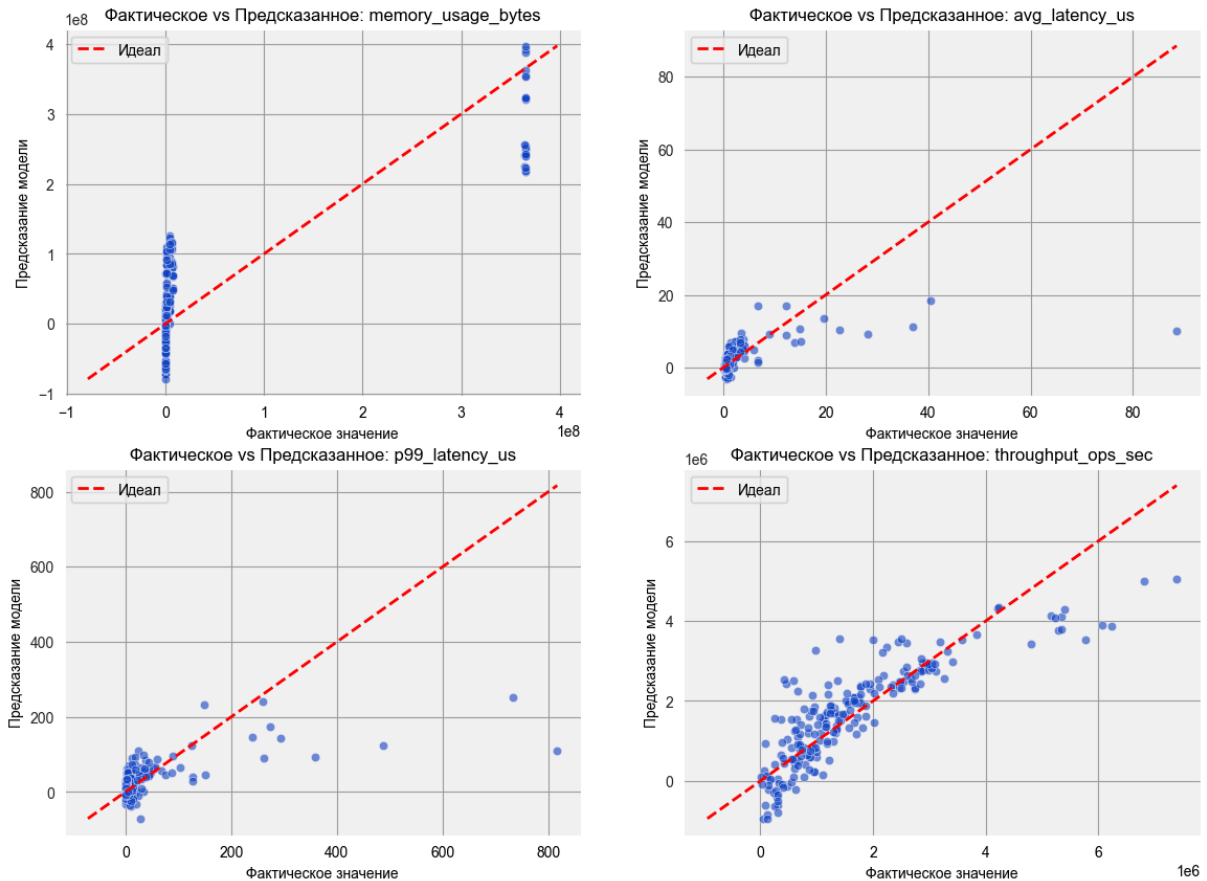
Оценка модели линейной регрессии:

Метрика	R^2	MAE	RMSE	MAPE	Max Error %
$memory_usage_bytes$	0.7348	44.4 млн	58.9 млн	52387 %	925633 %
$avg_latency_us$	0.3011	2.30	6.43	164.4 %	765 %
$p99_latency_us$	0.3732	31.76	74.44	787.7 %	7146 %

throughput_ops_sec	0.7238	514 045	720 979	87.7 %	1993 %
--------------------	--------	---------	---------	--------	--------

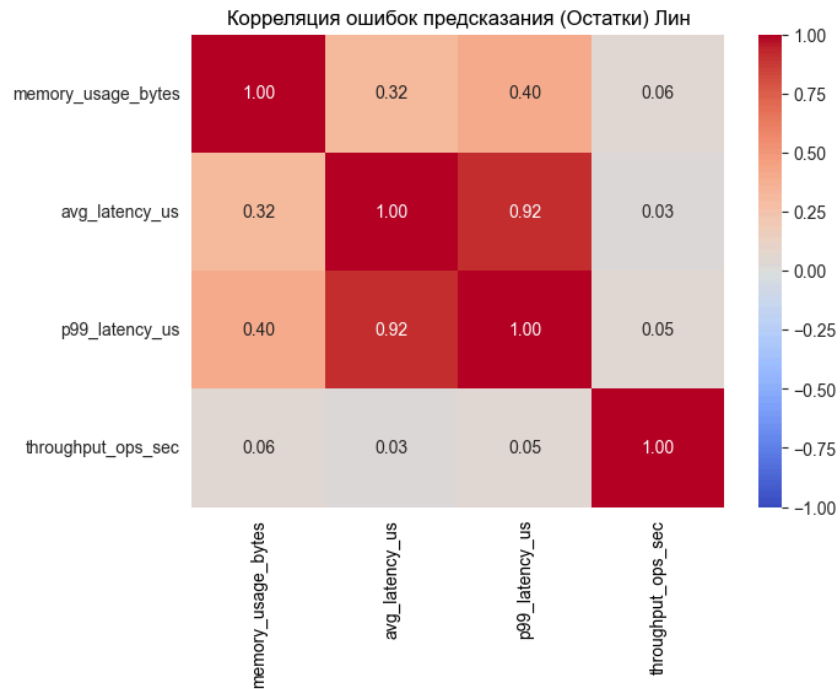
Линейная модель Ridge показала неоднородное качество: throughput_ops_sec преодолела порог достаточности 0.7, что объясняется прямой линейной зависимостью пропускной способности от предсказанной на первом звене средней латентности. memory_usage_bytes также преодолела порог - память задаётся структурными параметрами и относительно хорошо аппроксимируется линейной комбинацией логарифмов. Метрики латентности не преодолели порог достаточности.

Анализ графика «Фактическое vs Предсказанное»

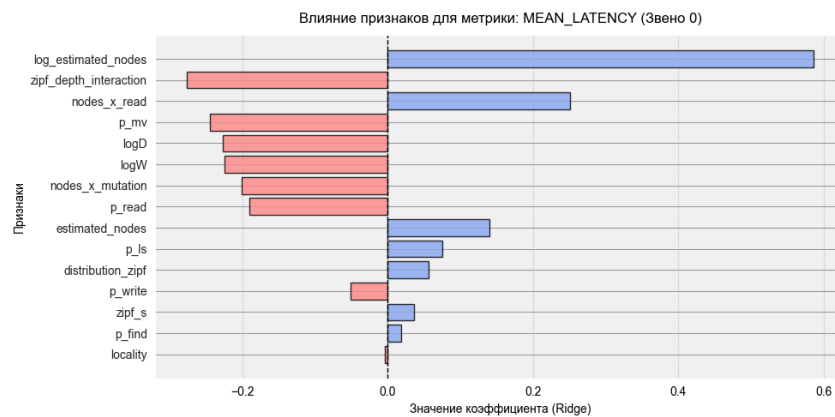


Для всех временных метрик (avg_latency, p99, throughput) наблюдается общий паттерн: в области высоких нагрузок точки систематически лежат ниже идеальной диагонали, что означает сильное занижение прогнозов моделью. Метрика памяти демонстрирует иной вид ошибки - стабильную переоценку на малых объёмах данных и недооценку на больших масштабах

Анализ остатков и обоснование перехода на Random Forest



Сильная корреляция остатков между `avg_latency_us` и `p99_latency_us` ($r = 0.92$) указывает на общий для обеих метрик неучтённый источник дисперсии. Ошибки памяти также умеренно связаны с задержками ($r = 0.32$ и $r = 0.40$), в то время как ошибки throughput практически изолированы



Анализ весов показал конфликт знаков у функционально связанных признаков. Для `avg_latency_us` доминирует `log_estimated_nodes` с весом $+0.58$, однако `logD` (-0.21) и `logW` (-0.20) получили крупные отрицательные коэффициенты - оба признака математически входят в формулу `estimated_nodes`, поэтому Ridge не может однозначно распределить вклад и вынужден компенсировать завышенный вес `log_estimated_nodes` отрицательными весами его компонент. Физически это означает, что модель «считает» рост глубины и ширины дерева снижающим латентность, что противоречит алгоритмической сложности $O(D)$.

Линейное приближение признано несостоятельным для латентностных метрик из-за нелинейной природы исследуемых процессов, в связи с чем осуществлён переход к нелинейным ансамблям

Фиксация основной модели (Random Forest)

Оптимизация гиперпараметров выполнялась методом `RandomizedSearchCV` (20 кандидатов, 5-блочная KFold, метрика R^2).

Лучшая конфигурация:

`n_estimators=200`,

`max_depth=12`,

`max_features=0.33`,

`min_samples_leaf=1`

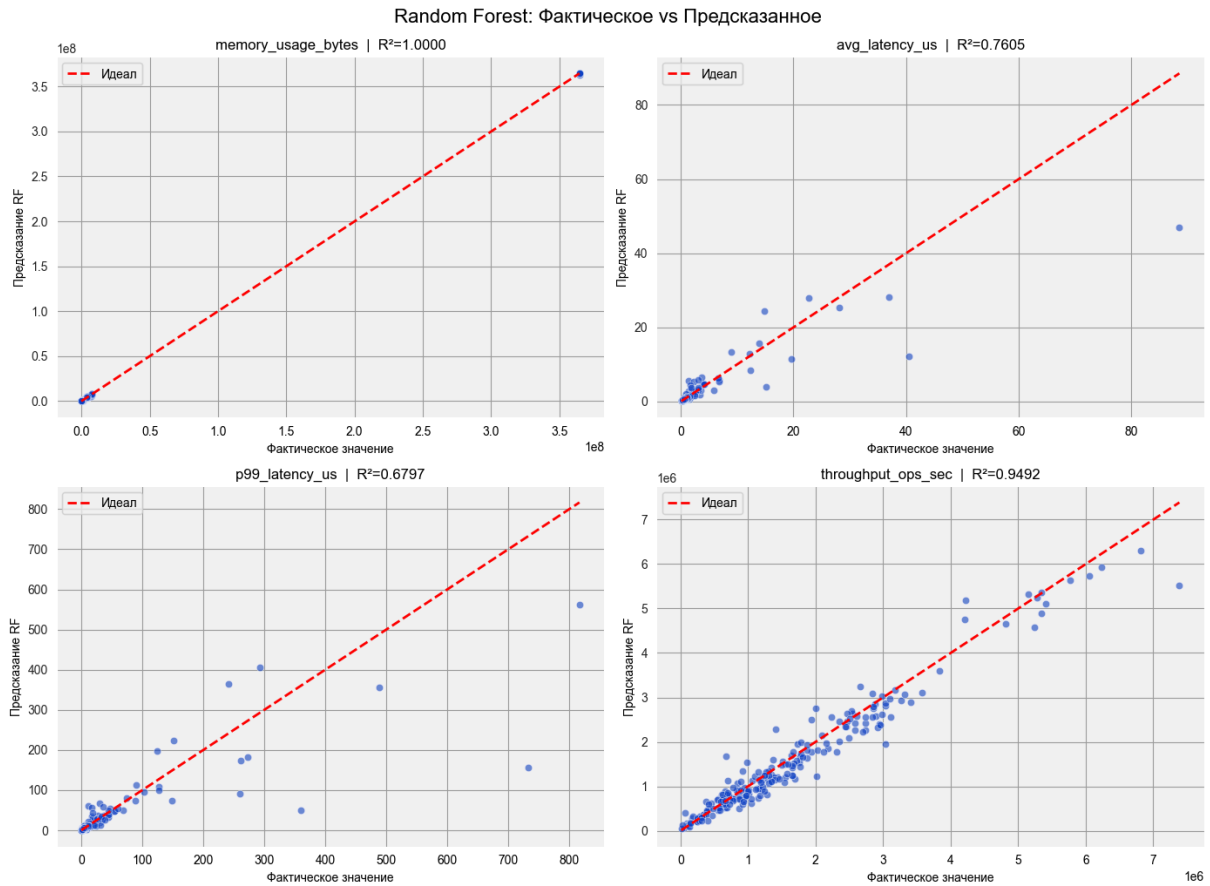
(средний кросс-валидационный $R^2 = 0.7566$, OOB $R^2 = 0.5625$ для стартового звена).

Оценка модели:

Метрика	R^2 Ridge	R^2 RF	MAE RF	RMSE RF	MAPE RF
<code>memory_usage_bytes</code>	0.7348	1.0000	103 007	271 007	18.19 %
<code>avg_latency_us</code>	0.3011	0.7605	0.87	3.76	22.80 %
<code>p99_latency_us</code>	0.3732	0.6797	12.82	53.21	35.33 %
<code>throughput_ops_sec</code>	0.7238	0.9492	211 101	309 070	22.39 %

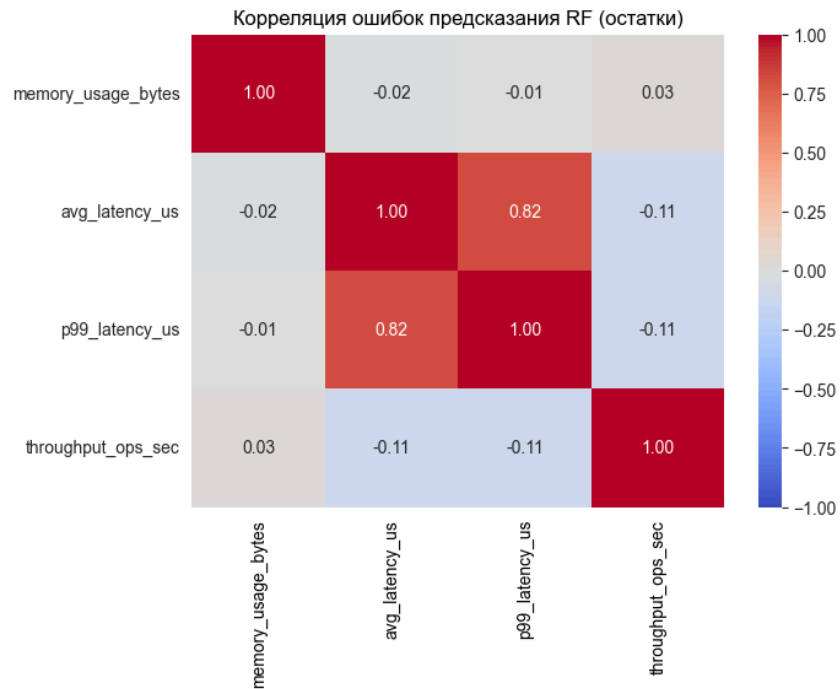
Переход к Random Forest дал качественный скачок точности по всем метрикам. Для `memory_usage_bytes` - идеальный $R^2 = 1.0000$ при MAPE всего 18.2%: лес восстанавливает детерминированную функцию через `estimated_nodes`. Для `avg_latency_us` прирост с 0.30 до 0.76 - наибольший относительный рост из всех метрик, MAPE снизилась с 164% до 22.8%. Для `throughput_ops_sec`, несмотря на то что Ridge уже преодолевал порог 0.7 (0.7238), RF добавил ещё +22.5 п.п., доведя R^2 до 0.9492 и снизив MAPE с 87.7% до 22.4%. Единственная метрика, где порог 0.7 не достигнут и у RF - `p99_latency_us` (0.6797), однако прирост с 0.37 почти вдвое подтверждает, что ограничивающий фактор здесь - не нехватка признакового сигнала, а нелинейность хвостового распределения, которую лес улавливает значительно лучше Ridge, хотя и не полностью.

Анализ графика «Фактическое vs Предсказанное»



Память воспроизводит идеальную линию во всём диапазоне. По `avg_latency_us` основная масса наблюдений прилегает к диагонали; единственная выраженная точка-выброс (фактическое ≈ 88 мкс, предсказание ≈ 47 мкс) - следствие структурного ограничения любого древесного ансамбля: модель не умеет экстраполировать за пределы обучающего диапазона и занижает прогноз на аномально высоких значениях. `throughput_ops_sec` показывает плотное прилегание к диагонали вплоть до 7×10^6 опер/с без систематического смещения. По `p99_latency_us` основная масса точек лежит вблизи идеала, однако два экстремальных наблюдения в верхней части диапазона предсказаны с занижением - тот же хвостовой эффект, что описан для `avg_latency`.

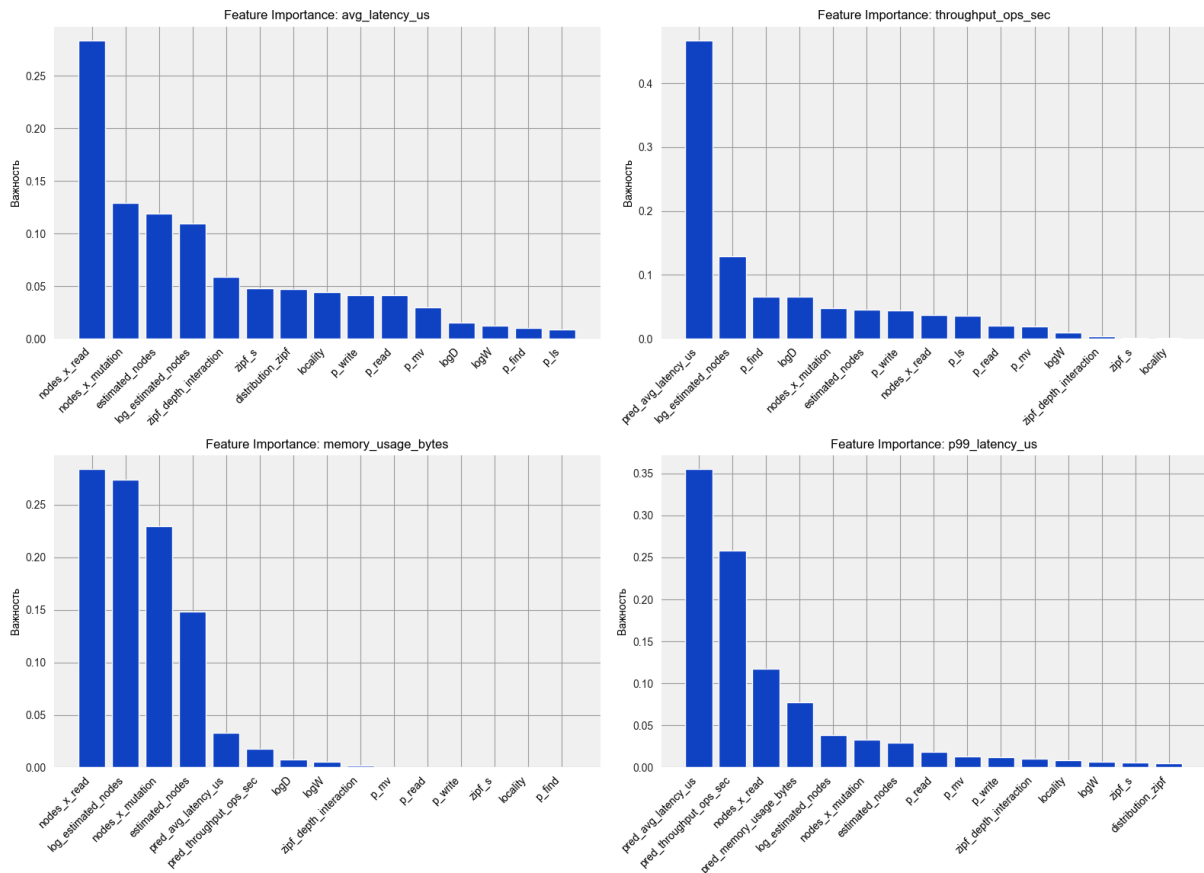
Анализ остатков



Корреляция остатков `avg_latency_us` и `p99_latency_us` снизилась с $r = 0.92$ (Ridge) до $r = 0.82$, подтверждая, что нелинейная модель частично развязала совместную дисперсию ошибок. Остатки `throughput_ops_sec` сохраняют умеренную отрицательную связь с латентностями ($r = -0.11$) - ошибки пропускной способности систематически противоположны по знаку ошибкам латентности, что является ожидаемым следствием физической обратной зависимости $\text{throughput} \approx 1/\text{latency}$. Остатки памяти полностью изолированы ($|r| \leq 0.03$) - ошибка памяти не коррелирует ни с одной другой метрикой.

Анализ важности признаков: извлечённые паттерны

Random Forest: Важность признаков по звеньям цепочки



- **avg_latency_us:** доминируют nodes_x_read (≈ 0.28) и nodes_x_mutation (≈ 0.13) - взаимодействие масштаба структуры с типом операции определяет более 40% важности. Это подтверждает гипотезу о том, что латентность в подходе А зависит не от размера дерева самого по себе, а от того, как размер сочетается с характером выполняемых операций (обход при чтении и при мутации имеет разную алгоритмическую стоимость). distribution_zipf (≈ 0.05) и locality (≈ 0.05) вносят умеренный, но ненулевой вклад - неравномерность доступа влияет на латентность как вторичный эффект
- **throughput_ops_sec:** pred_avg_latency_us безоговорочно доминирует (≈ 0.47), количественно фиксируя физическую обратную связь между задержкой и пропускной способностью. Вторичный блок - log_estimated_nodes (≈ 0.13), p_find (≈ 0.08), logD (≈ 0.07) и nodes_x_mutation (≈ 0.06) - совокупно даёт около 35% важности, показывая, что конкретный тип поисковых операций влияет на throughput независимо от латентности
- **memory_usage_bytes:** важность почти целиком сосредоточена на структурных признаках - nodes_x_read (≈ 0.28), log_estimated_nodes (≈ 0.27), nodes_x_mutation (≈ 0.23) и estimated_nodes (≈ 0.15) дают суммарно около 93%. Признаки предсказанных метрик вносят менее 6% - память является чисто структурной величиной, не зависящей от операционного профиля
- **p99_latency_us:** pred_avg_latency_us (≈ 0.35) и pred_throughput_ops_sec (≈ 0.26) доминируют, далее идут nodes_x_read (≈ 0.11) и pred_memory_usage_bytes (≈ 0.08). Совокупный вклад предсказаний предыдущих звеньев (≈ 0.69) существенно превышает вклад сырых признаков (0.31)

Анализ подхода В

Подход В сохраняет дерево как основу, но добавляет хэш-таблицу «полный путь → узел». Это обеспечивает $O(1)$ -доступ при прямой адресации, но усложняет операцию перемещения (mv), требующую синхронизации обеих структур

Инженерия признаков

Для подхода В сформирован набор из 12 признаков: геометрические параметры (logD, logW), полный вероятностный профиль операций без агрегации (p_ls, p_find, p_write, p_mv, p_read), локальность доступа (locality), параметры распределения Ципфа (zipf_s, zipf_depth_interaction) и оценка масштаба (estimated_nodes, log_estimated_nodes).

Расчёт VIF показал характерную картину коллинеарности:

Признак	VIF	Признак	VIF
p_ls	15.10	zipf_depth_interaction	6.73
p_read	12.46	locality	5.89
zipf_s	9.70	log_estimated_nodes	5.67
p_find	8.76	logW	3.35
p_write	8.02	p_mv	3.18
logD	7.31	estimated_nodes	1.88

Критический порог ($VIF > 10$) превышают только p_ls и p_read. Это вызвано не дублированием параметров, а функциональной зависимостью. Параметр объема estimated_nodes ($VIF = 1.88$) здесь практически ортогонален остальным, в отличие от подхода А

Фиксация базовой модели (Ridge)

Кросс-валидация снова выбрала максимальную регуляризацию ($\alpha = 0.8$), что ожидаемо при текущей коллинеарности. Порядок цепочки полностью совпадает с подходом А, упрощая сравнение: avg_latency_us → throughput_ops_sec → memory_usage_bytes → p99_latency_us.

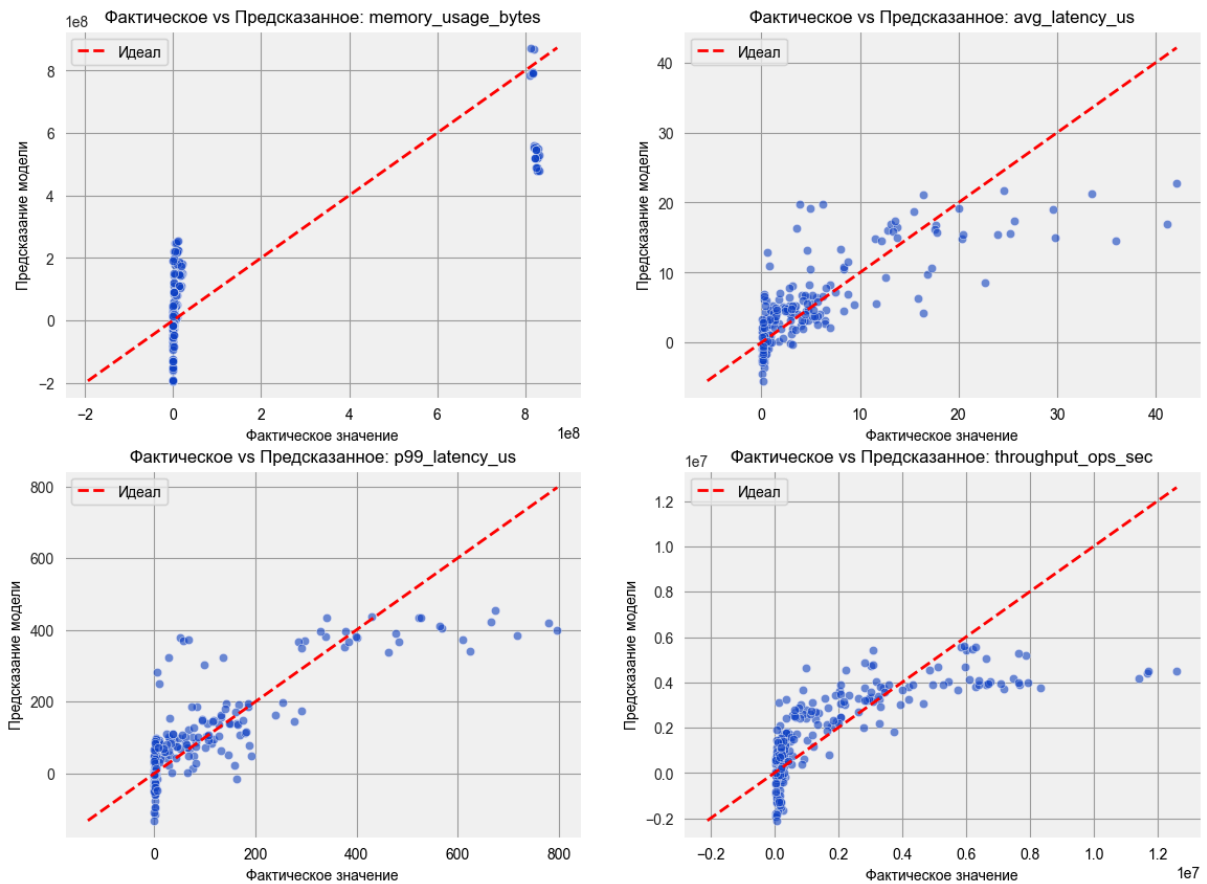
Оценка модели

Метрика	R^2	MAE	RMSE	MAPE	Max Error %
memory_usage_bytes	0.7459	98.66 млн	134.77 млн	79301.5 %	1061332 %
avg_latency_us	0.6034	3.09	4.83	376.1 %	5341 %
p99_latency_us	0.6735	62.45	94.27	2920.4 %	65920 %
throughput_ops_sec	0.5538	1.22 млн	1.73 млн	396.4 %	4745 %

Линейная регрессия Ridge показала недостаточное качество ($R^2 < 0.7$ для временных метрик). Однако результаты avg_latency_us (0.6034) и p99_latency_us (0.6735) почти вдвое лучше, чем у подхода А.

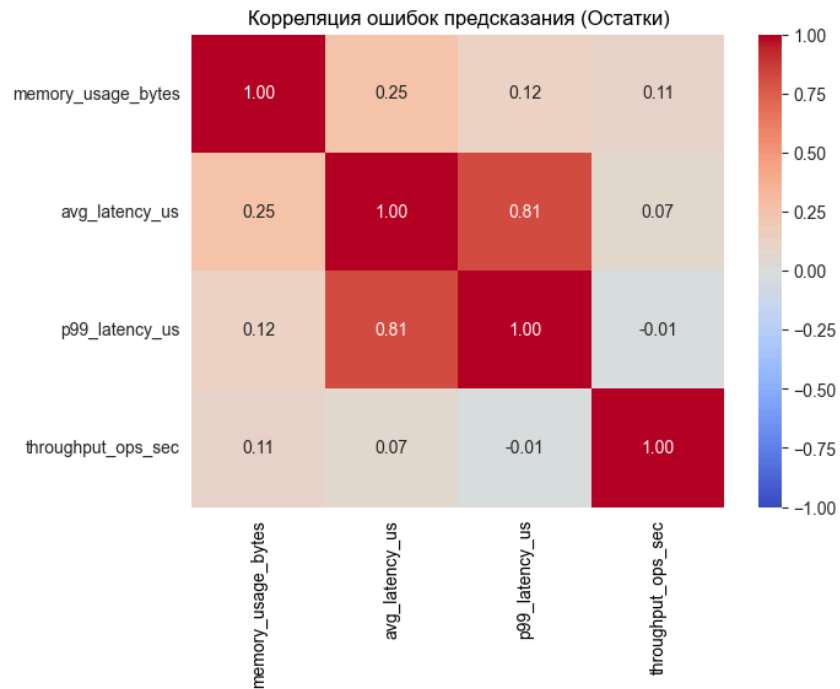
Хэш-таблица убирает зависимость стоимости поиска от глубины дерева D , делая связь латентности со структурой более линейной. Высокая относительная ошибка (MAPE) указывает на неспособность Ridge учитывать пороговые эффекты при росте estimated_nodes

Анализ графика «Фактическое vs Предсказанное»

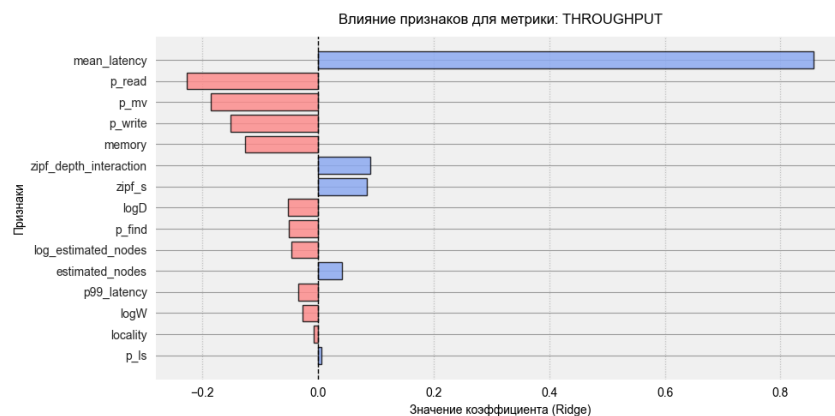
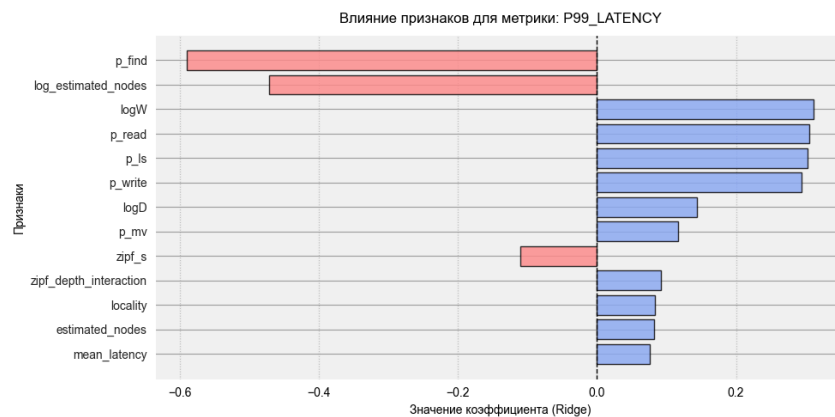


- **Память:** Модель воспроизводит бимодальный паттерн (кластеры около 0 и $8 \cdot 10^8$), но внутри верхнего кластера уходит в регрессию к среднему, разделяя предсказания на две группы.
- **Латентность и пропускная способность:** Наблюдается системное «срезание» пиковых значений. Модель принимает экспоненциальный рост за линейный тренд, но в подходе В плато выходит на более высокий уровень, то есть точность теряется позже, чем в А

Анализ остатков и обоснование перехода на Random Forest



Анализ корреляции остатков выявил сильную связь между средней и хвостовой латентностью ($r = 0.81$). Линейный оператор Ridge не может изолировать общий нелинейный источник дисперсии (кэширование, задержки хэш-таблицы). Ошибки пропускной способности (throughput_ops_sec) почти не коррелируют с остальными метриками



- **p99_latency_us**: Доминирует `log_estimated_nodes` (+1.25), которому физически бессмысленно противодействуют `logD` (-0.95) и `logW` (-0.45) - признак нестабильности весов
- **throughput_ops_sec**: Главный вклад вносит `mean_latency` (+0.85), отражая обратную связь латентности и пропускной способности. Вероятности `p_read` (-0.25), `p_mv` (-0.17) и `p_write` (-0.15) снижают throughput сильнее, чем в А, из-за расходов на синхронизацию структур при mv

Линейная гипотеза признается несостоятельной, что оправдывает переход к Random Forest

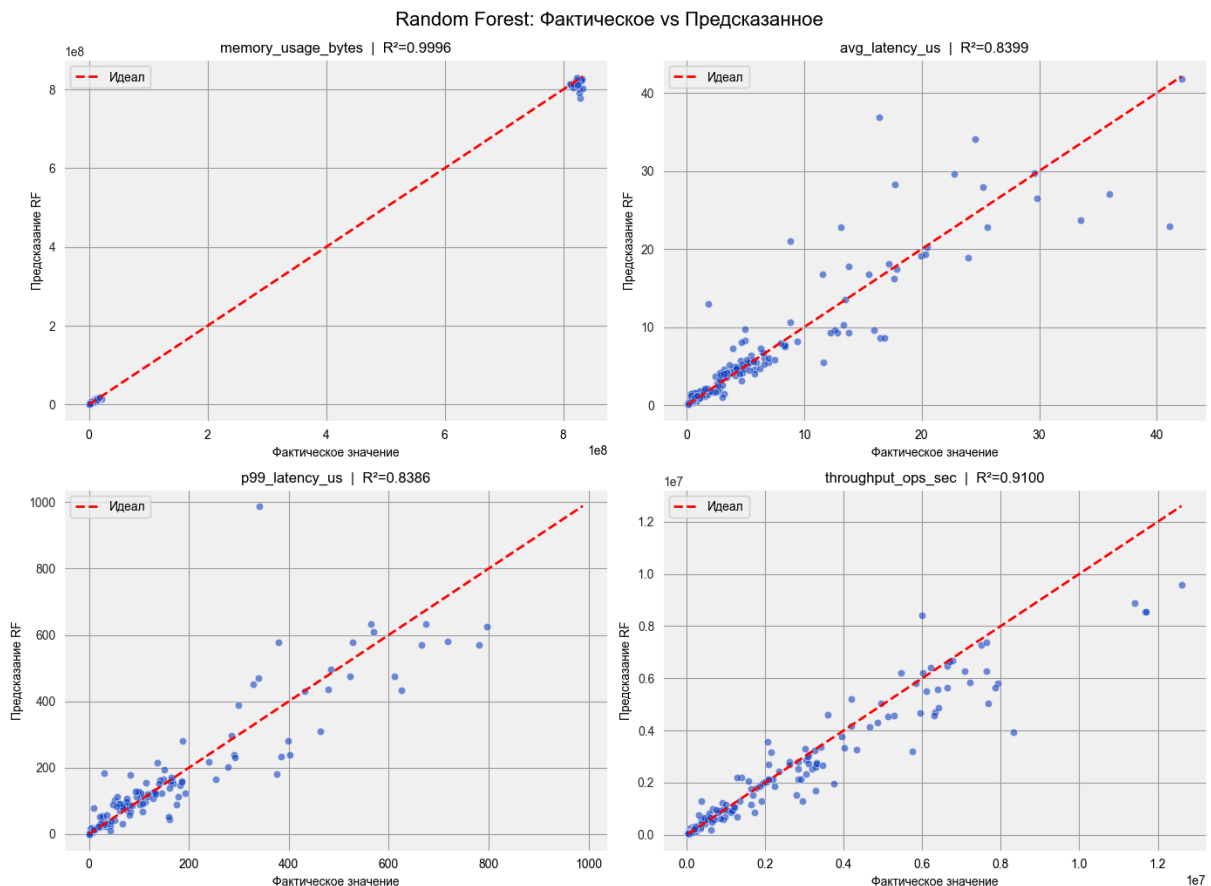
Random Forest: подбор гиперпараметров и оценка

Оптимальная конфигурация леса: `n_estimators=200`, `max_depth=10`, `max_features=sqrt`, `min_samples_leaf=1`. Менее консервативные настройки (по сравнению с А) указывают на более слабую мультиколлинеарность признаков

Метрика	R^2	MAE	RMSE	MAPE
memory_usage_bytes	0.9996	1 618 333	5 450 234	181.09 %
avg_latency_us	0.8399	1.27	3.07	32.87 %
p99_latency_us	0.8386	27.33	66.27	54.18 %
throughput_ops_sec	0.9100	377 639	776 423	25.85 %

Случайный лес дает резкий прирост качества. В отличие от подхода А, все четыре метрики уверенно преодолевают порог $R^2 = 0.7$. Подход В оказывается более предсказуемым для ансамблей: $O(1)$ -доступ сглаживает критические нелинейности

Анализ графика «Фактическое vs Предсказанное» (RF)



- **Память:** Идеальное прилегание к линии
- **Латентность и throughput:** Плотное распределение вдоль диагонали с небольшим занижения.
- **p99_latency_us:** Обнаружено одно аномальное предсказание: факт ≈ 350 мкс предсказан как ≈ 990 мкс. Такой выброс в сторону переоценки уникален для подхода В и вызван редкой комбинацией признаков

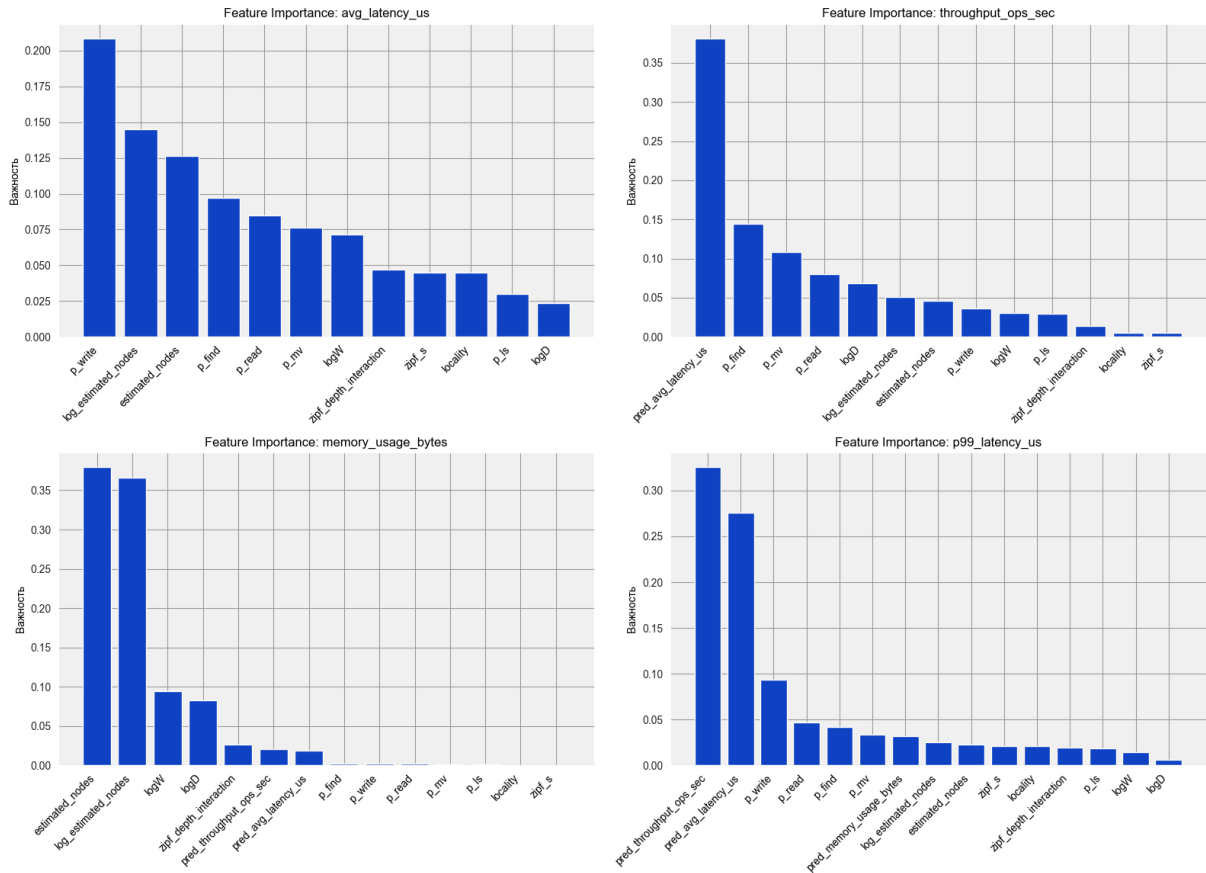
Анализ остатков



Остаточная корреляция между avg_latency_us и p99_latency_us осталась высокой ($r = 0.77$). Так как этот паттерн повторяет результат подхода А, его причина это ограничение цепочечной структуры RF (модель p99 неизбежно наследует нескорректированную ошибку предыдущего звена)

Важность признаков

Random Forest: Важность признаков по звеньям цепочки



- **avg_latency_us**: Доминирует доля операций записи `p_write` (≈ 0.21), за ней идут `log_estimated_nodes` (≈ 0.145) и `estimated_nodes` (≈ 0.125). Модель сама выявила накладные расходы на синхронизацию хэш-таблицы при записи.
- **throughput_ops_sec**: Главный признак - `pred_avg_latency_us` (≈ 0.38). Также важны операции `p_find` (≈ 0.145) и `p_mv` (≈ 0.105), напрямую нагружающие хэш-таблицу.
- **memory_usage_bytes**: Более 70% важности делят `estimated_nodes` и `log_estimated_nodes`.
- **p99_latency_us**: В отличие от A, здесь `pred_throughput_ops_sec` (≈ 0.325) важнее, чем `pred_avg_latency_us` (≈ 0.275). Конкурентный доступ к хэш-таблице сначала вызывает просядку пропускной способности, которая затем транслируется в хвостовые задержки (p99)

Анализ подхода С

Подход С полностью отказывается от древовидной структуры в оперативной памяти, организуя файловую систему в виде плоской хэш-таблицы. Это гарантирует константный доступ $O(1)$ на чтение и запись в любой точке, исключая необходимость вложенных обходов, но превращает структурные мутации в тяжелую операцию $O(K)$, где K - количество всех вложенных подфайлов.

Инженерия признаков

Признаковое пространство С объединяет подходы А и В: сохранены перекрестные взаимодействия `nodes_x_read` и `nodes_x_mutation` (из А) и добавлены отдельные вероятности операций `p_ls`, `p_find`, `p_write`, `p_mv`, `p_read` (из В). Из набора полностью исключен параметр `logD`, что сократило размерность до 11 признаков.

Расчёт VIF показал наилучшую ортогональность среди всех конфигураций:

Признак	VIF	Признак	VIF
estimated_nodes	13.80	nodes_x_read	4.26
p_find	7.30	p_read	3.53
nodes_x_mutation	6.39	p_mv	2.60
p_ls	5.11	log_estimated_nodes	1.43
p_write	4.74	locality / logW	≈0.96-0.98

Порог $VIF > 10$ превышен только для estimated_nodes. Несмотря на минимальную коллинеарность, линейная модель демонстрирует наихудшие результаты среди всех систем. Это доказывает, что главным ограничением здесь выступает экстремальная нелинейность зависимостей, а не избыточность признаков.

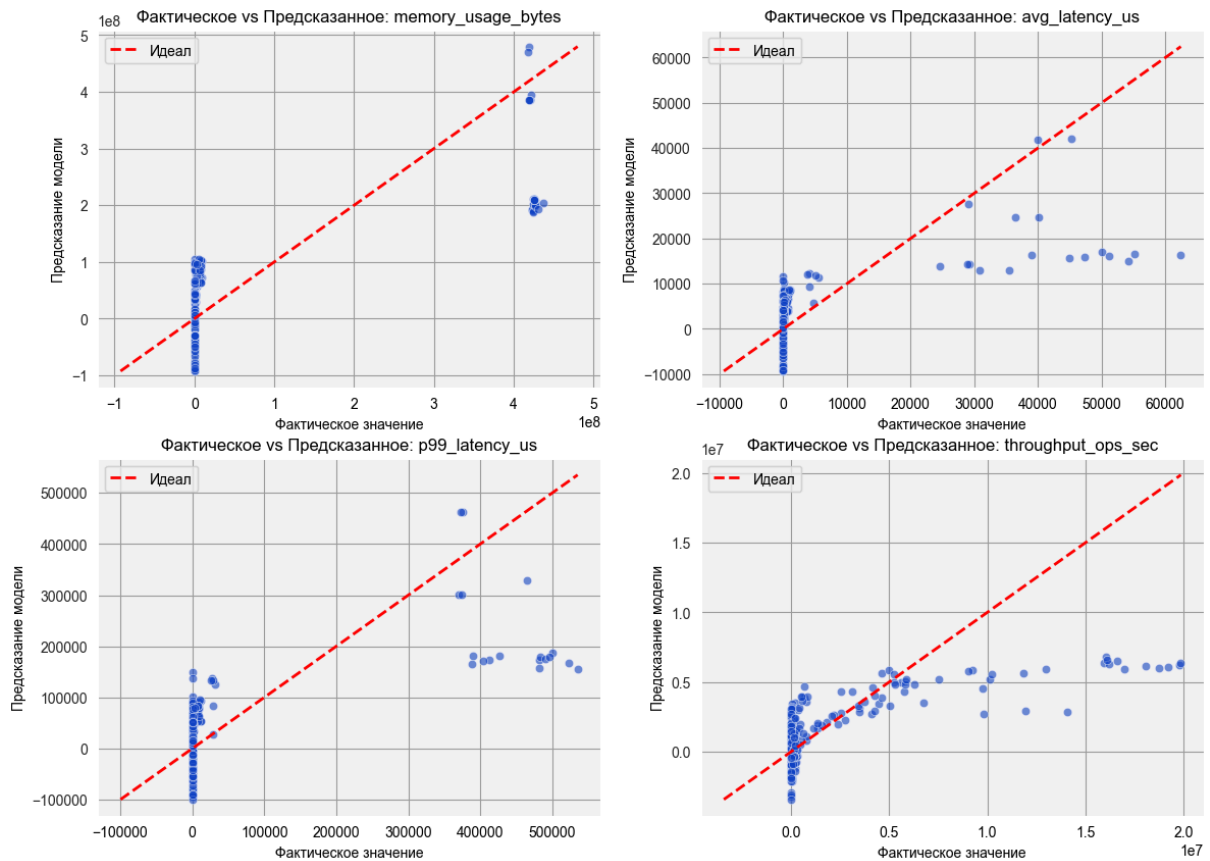
Фиксация базовой модели (Ridge) и оценка качества

Оптимальная конфигурация Ridge: $\alpha = 0.8$, порядок звеньев цепочки: avg_latency_us → throughput_ops_sec → memory_usage_bytes → p99_latency_us (совпадает с порядком RF для подхода A).

Метрика	R^2	MAE	RMSE	MAPE	Max Error %
memory_usage_bytes	0.6134	64.92 млн	85.73 млн	368 844 %	9 866 479 %
avg_latency_us	0.4653	5403.56	8629.28	1 066 598 %	15 501 440 %
p99_latency_us	0.4749	60 062.83	89 309.91	5 143 905 %	108 288 436 %
throughput_ops_sec	0.4672	1.89 млн	3.27 млн	637 796 %	18 881 916 %

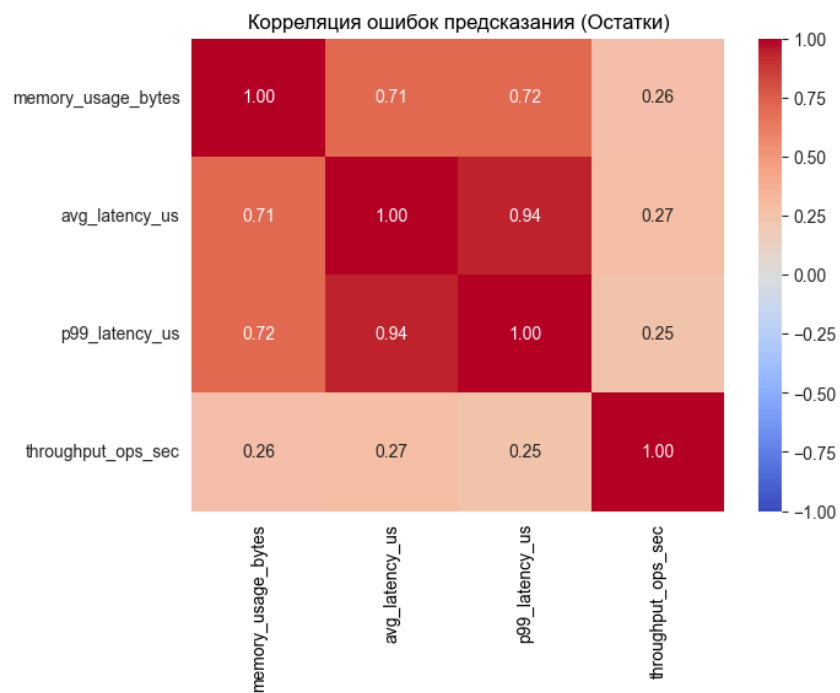
Все метрики находятся ниже порога $R^2 = 0.7$. В отличие от подходов A и B, Ridge в системе C не справился даже с аппроксимацией памяти ($R^2 = 0.6134$). Абсолютный масштаб задержек (десятки и сотни миллисекунд) указывает на присутствие в выборке редких, но тяжелых операций, генерирующих аномальные значения MAPE и Max Error %

Анализ графика «Фактическое vs Предсказанное»

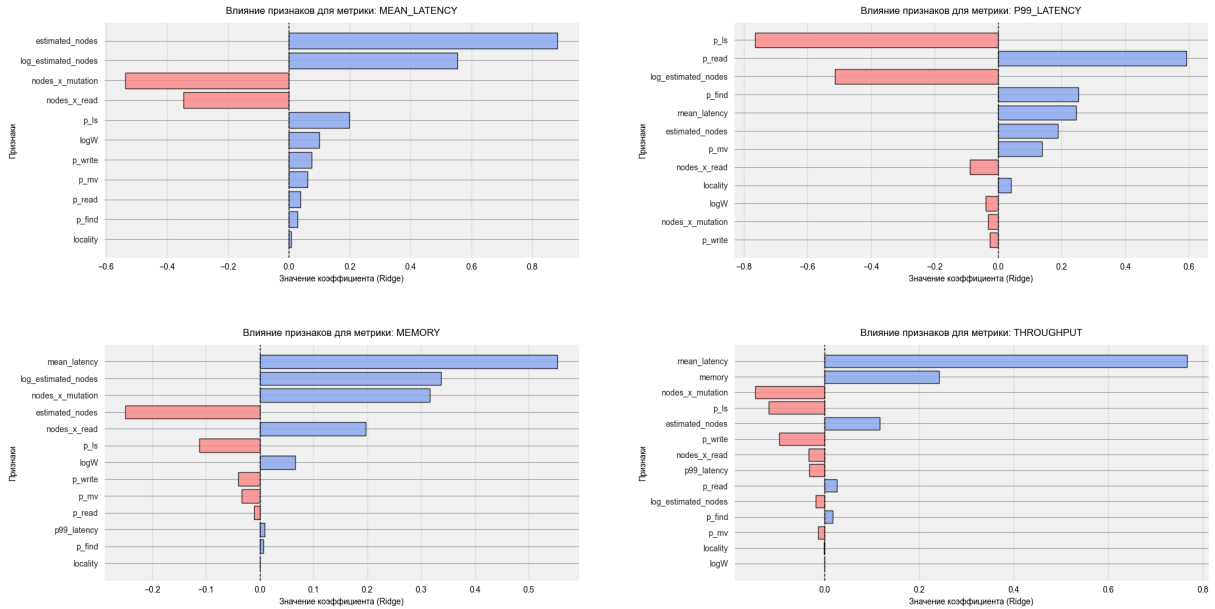


На графике throughput_ops_sec предсказания Ridge выходят на плато, полностью срезая реальные пиковые значения.

Анализ остатков и обоснование перехода на Random Forest



Структура остатков принципиально отличается от А и В: впервые зафиксирована сильная корреляция ошибок `memory_usage_bytes` с латентностями ($r \approx 0.72$, тогда как в А/В связь стремилась к нулю). Взаимосвязь остатков `avg_latency` и `p99` достигла максимума ($r = 0.94$). Это указывает на единый нелинейный источник дисперсии (аномальные пиковые нагрузки), при котором одновременно деградируют временные показатели и скачкообразно растет объем структуры



- **avg_latency_us:** Наблюдается знакомый по А конфликт знаков: `estimated_nodes` (+0.89) против отрицательных весов интерактивных признаков (`nodes_x_mutation`: -0.55)
- **throughput_ops_sec:** Сильнейшим негативным фактором выступает доля листинга `p_ls` (-0.75)
- **memory_usage_bytes:** Обнаружена важная аномалия: доминирующим предиктором памяти стал не геометрический параметр, а предсказанная латентность `mean_latency` (+0.55). Потребление памяти в С динамически зависит от временных задержек файловой системы (например, накопление буферов при долгом ответе)
- **p99_latency_us:** Модель опирается исключительно на предсказания предыдущих шагов: `mean_latency` (+0.77) и `memory` (+0.25)

Провал линейной регрессии на фоне чистых признаков подтверждает необходимость перехода к Random Forest для раскрытия сложных пороговых эффектов

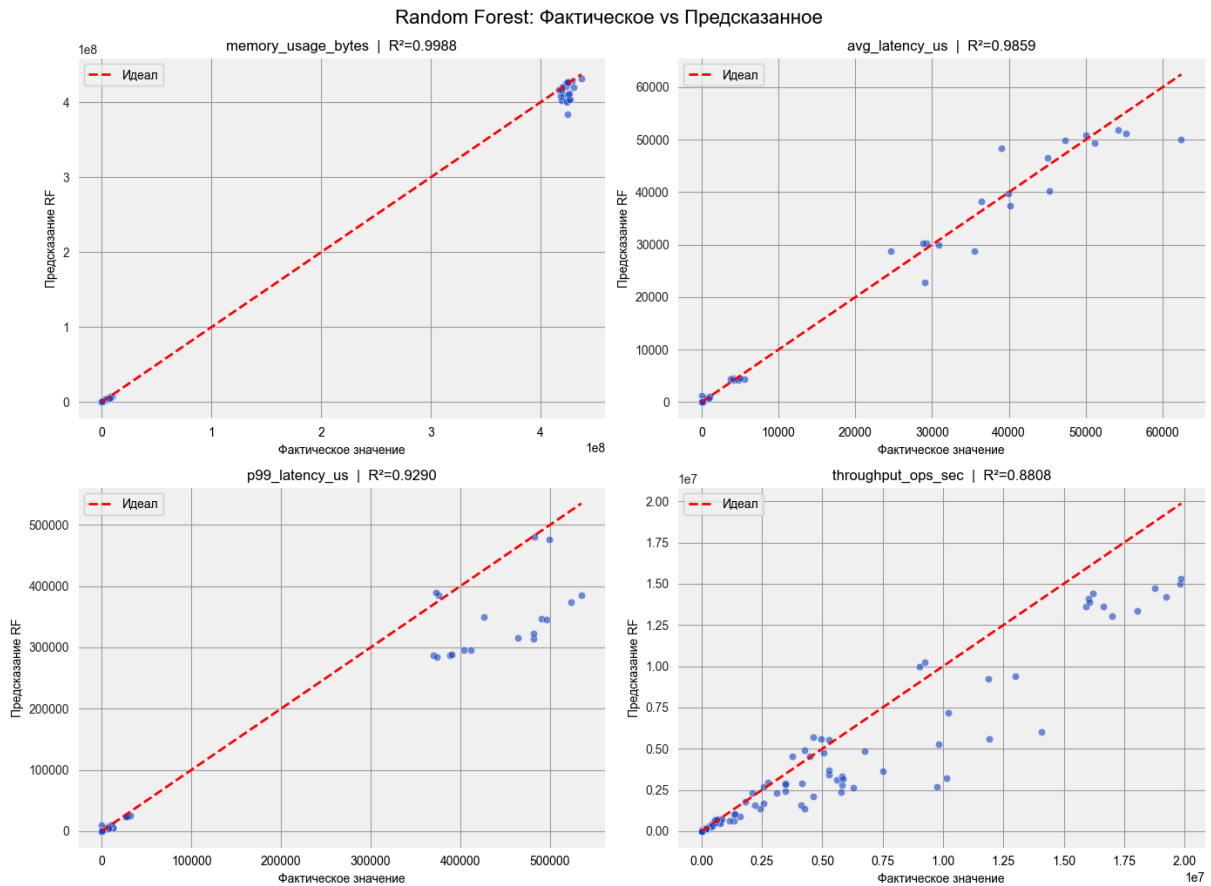
Random Forest: подбор гиперпараметров и оценка

Оптимальная конфигурация: `n_estimators=200`, `max_depth=10`, `max_features=sqrt`, `min_samples_leaf=1`. Порядок цепочки сохранен. OOB R^2 первого звена (`avg_latency_us`) составил 0.9919, подтверждая высокую обобщающую способность

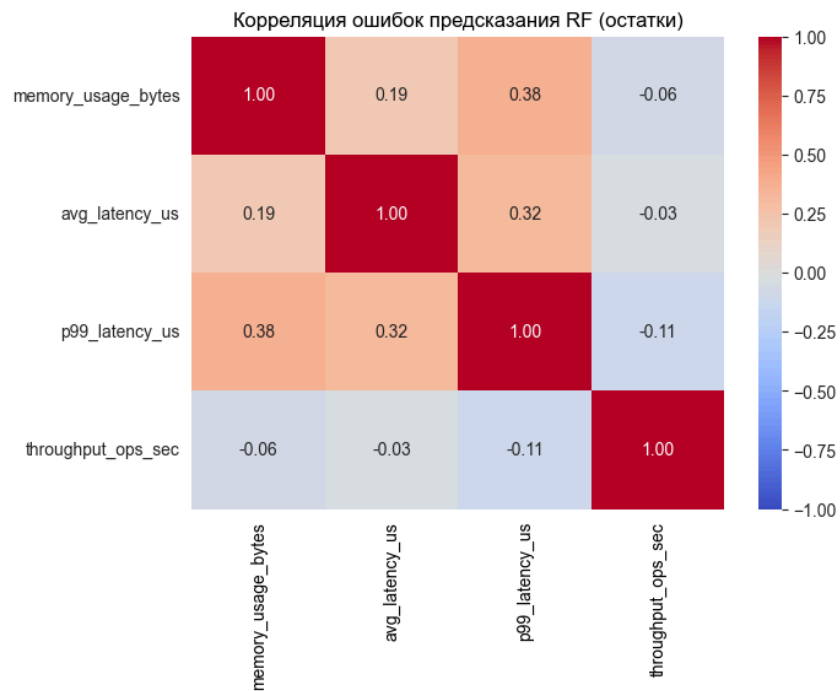
Метрика	R^2	MAE	RMSE	MAPE
memory_usage_bytes	0.9988	1 313 819	4 743 374	101.01 %
avg_latency_us	0.9859	338.82	1 402.37	2533.07 %
p99_latency_us	0.9290	8 797.01	32 839.16	2595.51 %
throughput_ops_sec	0.8808	627 442	1 547 501	579.56 %

Переход к ансамблям продемонстрировал самый сильный скачок качества среди всех подходов. Подход С на базе RF стал лидером по точности аппроксимации временных параметров, существенно опередив и А, и В

Анализ графика «Фактическое vs Предсказанное» (RF)



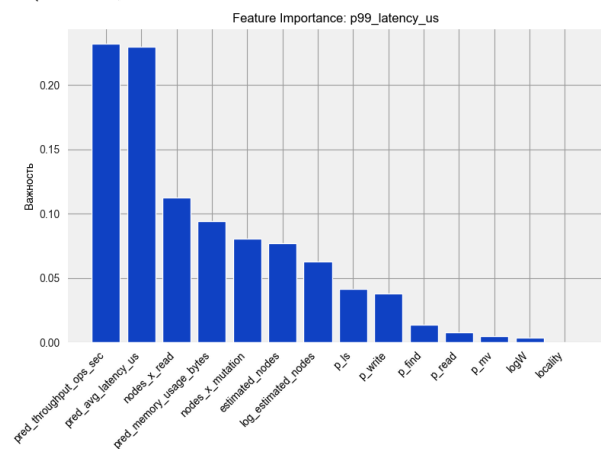
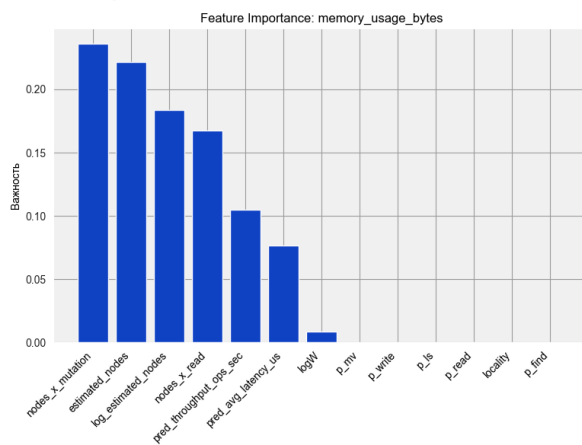
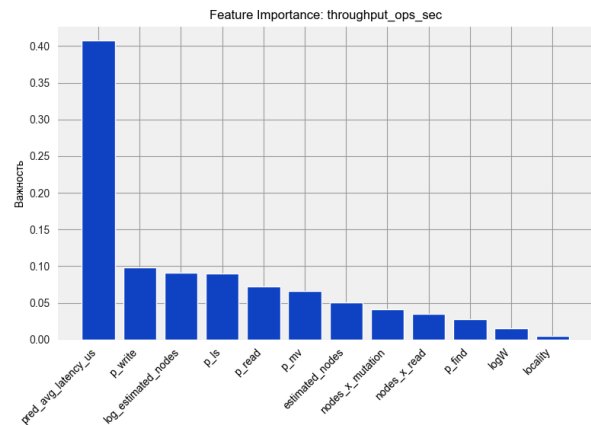
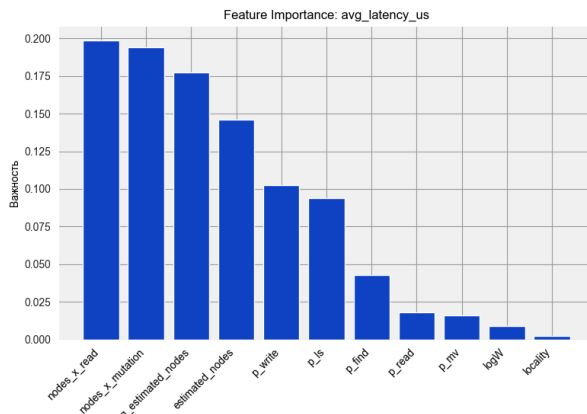
Точки `avg_latency_us` прилегают к диагонали до максимумов, эффект насыщения устранен. На `p99` и `throughput` сохраняются занижения на больших значениях, но куда меньше чем в линейной модели

Анализ остатков

Корреляция остатков `avg_latency` / `p99_latency` упала с 0.94 до 0.32 (что существенно чище результатов RF в подходах А и В, где оставалось около 0.8). Единственной значимой парой осталась взаимосвязь `memory` / `p99_latency` ($r = 0.38$), что обусловлено структурой цепочки

Важность признаков: извлеченные паттерны

Random Forest: Важность признаков по звеньям цепочки



- **avg_latency_us**: Важность распределена между интерактивными признаками nodes_x_read (≈ 0.20), nodes_x_mutation (≈ 0.19) и геометрией (log_estimated_nodes ≈ 0.18). Для хэш-таблицы это затраты на эмуляцию иерархии через префиксы: стоимость префиксного поиска при чтении и массового пересчета ключей-путей при мутациях прямо масштабируется от объема структуры
- **throughput_ops_sec**: Наблюдается тотальное доминирование признака pred_avg_latency_us (≈ 0.41).
- **memory_usage_bytes**: RF исправил аномалию Ridge. Память снова объясняется структурными факторами: nodes_x_mutation (≈ 0.24) и estimated_nodes (≈ 0.22).
- **p99_latency_us**: Демонстрирует наиболее сложную композитную структуру важности. Метрика в равной степени зависит от pred_throughput_ops_sec (≈ 0.23) и pred_avg_latency_us (≈ 0.23), со значимым влиянием nodes_x_read (≈ 0.11) и pred_memory_usage_bytes (≈ 0.09).

Форма связи между метриками (объединённая выборка)

Анализ на объединённых данных всех подходов проверяет характер связей между метриками: являются ли они линейными или нелинейными монотонными.

Диагностика: Пирсон vs Спирмен

Коэффициент Пирсона оценивает силу только линейной связи. Коэффициент Спирмена рассчитывается по рангам значений и равен ровно 1 (или -1) для любой строго монотонной зависимости (логарифмической, гиперболической и т.д.). Существенный разрыв между ними это маркер сильной нелинейной связи.

Пара метрик	Pearson	Spearman	Разрыв	R^2 лин.	R^2 квадр.
avg_latency vs p99_latency	0.953	0.971	0.018	0.908	0.960
avg_latency vs memory	0.321	0.499	0.178	0.103	0.114
avg_latency vs throughput	-0.111	-0.998	0.887	0.012	0.015
p99_latency vs memory	0.327	0.472	0.144	0.107	0.112
p99_latency vs throughput	-0.113	-0.971	0.858	0.013	0.015
memory vs throughput	-0.169	-0.485	0.316	0.028	0.036

Анализ ключевых паттернов

- **Латентность vs Throughput (критическая нелинейность):** Наблюдается экстремальный разрыв между Пирсоном (≈ -0.11) и Спирменом (≈ -0.99). Связь почти детерминированная, но обратно пропорциональная. Линейная модель (Ridge) принимает эту гиперболу за случайный шум, чем объясняется её провал на throughput ($R^2 < 0.58$). Низкий квадратичный R^2 (0.015) доказывает неэффективность простых полиномов для аппроксимации этой формы связи.
- **avg_latency vs p99_latency (линейный полюс):** Разрыв минимален (0.018), линейная модель сразу объясняет 90.8% дисперсии. Метрики тесно связаны линейно, а их совместные отклонения вызваны внешними факторами (пиковыми нагрузками), а не формой взаимозависимости.
- **Память (структурная автономность):** Связь memory с обеими латентностями слабая в любой форме (Spearman $\approx 0.47 - 0.50$). Это подтверждает, что объём структуры определяется преимущественно статическими геометрическими параметрами, а не временной динамикой выполнения операций.